Laborator 2

Exerciții

1. Fie setul de date MNIST ce conține imagini cu cifre de la 0 la 9.
Completați metoda __getItem__(self, idx) a Dataset-ului astfel încât să returneze imaginea vectorizată și scalată în [0, 1] precum și label-ul asociat.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets
from torch.utils.data import Dataset, DataLoader

train_data = datasets.MNIST(root="./data", train=True, download=True)
test_data = datasets.MNIST(root="./data", train=False, download=True)

X_train = train_data.data.numpy()
y_train = train_data.targets.numpy()

X_test = test_data.data.numpy()
y_test = test_data.targets.numpy()

class MNISTVector(Dataset):
    def __init__(self, images, labels):
        self.images = images
        self.labels = labels

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        …


train_dataset = MNISTVector(X_train, y_train)
test_dataset  = MNISTVector(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=50, shuffle=True)
test_loader  = DataLoader(test_dataset, batch_size=50, shuffle=False)

for X, y in train_loader:
    print(X.shape, y.shape)
    break
```

2. Implementați o rețea simplă pentru a procesa imaginile vectorizate.
   - Rețeaua trebuie să conțină doar straturi complet conectate. (nn.Linear(...)).
   - Rețeaua trebuie să conțină cel puțin 2 straturi (inclusiv ultimul).
   - După ultimul strat nu urmează activare.
   - După orice alt strat puteți să folosiți fie activare sigmoid (F.sigmoid(...)) sau tanh (F.tanh(...)) sau relu (F.relu(...)).
   - Parametrii rețelei se scriu în constructor (self.fc1 = nn.Linear(...)).
   - Parcurgerea rețelei se scrie în metoda forward.
   - Intrarea rețelei este un tensor de dimensiunea (B, 784) unde B este batch_size și 784 reprezintă dimensiunea unui vector anterior calculat.
   - Ieșirea rețelei este un tensor de dimensiune (B, 10) unde B este batch_size și 10 reprezintă numărul de clase.

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        …

    def forward(self, x):
        …


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = MLP().to(device)

for X, y in train_loader:
    X, y = X.to(device), y.to(device)
    y_pred = model(X)
    print(y_pred.shape)
    break
```

3. Antrenați modelul și observați rezultatele. Influențează funcțiile de activare rezultatele? Dar numărul de neuroni?

```
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

def train(model, loader):
    model.train()

    total_loss = 0
    total_correct = 0
    total_data = 0

    for X, y in loader:
        X, y = X.to(device), y.to(device)

        optimizer.zero_grad()

        out = model(X)
        loss = criterion(out, y)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        total_correct += (out.argmax(dim=-1) == y).sum().item()
        total_data += y.size(0)

    return total_loss / len(loader), total_correct / total_data

epochs = 10
for epoch in range(epochs):
    train_loss, train_acc = train(model, train_loader)
    test_loss, test_acc = test(model, test_loader)
    print(f"Epoch {epoch}:")
    print(f"train loss={train_loss:.4f}, train acc={train_acc:.4f}")
```

4. Scrieti o functie similară pentru a calcula funcția de cost și acuratețea pe datele de testare și apelați-o după pasul de antrenare.

```
def test(model, loader):
    model.eval()

    with torch.no_grad():
        …
```

5. Pentru datele de antrenare, înainte de a antrena rețeaua, calculați media și deviația pentru fiecare din cele 784 trăsături, după ce au fost scalate. Folosiți vectorii de medie și deviație pentru a normaliza datele în clasa MNISTVector. Țineți cont de faptul că datele testare trebuie să parcurgă aceleași etape de pre procesare ca datele de antrenare.

6. După normalizarea de la 5., pentru datele de antrenare aplicați PCA pentru a obține 100 de trăsături. Țineți cont de faptul că datele testare trebuie să parcurgă aceleași etape de pre procesare ca datele de antrenare.

7. Aplicați PCA cu albire pentru a obține 100 de trăsături.

8. Antrenați o rețea cu cel puțin 2 straturi ascunse în care rezultatul primului strat ascuns contribuie la un strat mai avansat decât exact următorul.

9. Adăugați straturi de regularizare (nn.Dropout(...) sau nn.BatchNorm1d(...)) și antrenați rețeaua.

10. Fără a folosi deloc datele de testare în procesul de antrenare, antrenați un model cu cel mult 1 milion de parametri antrenabili compus doar din straturi complet conectate care obține o acuratețe de cel puțin 98% pe datele de testare.
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)